

서울과학기술대학교
SEOULTECH Seoul National University of Science & Technology

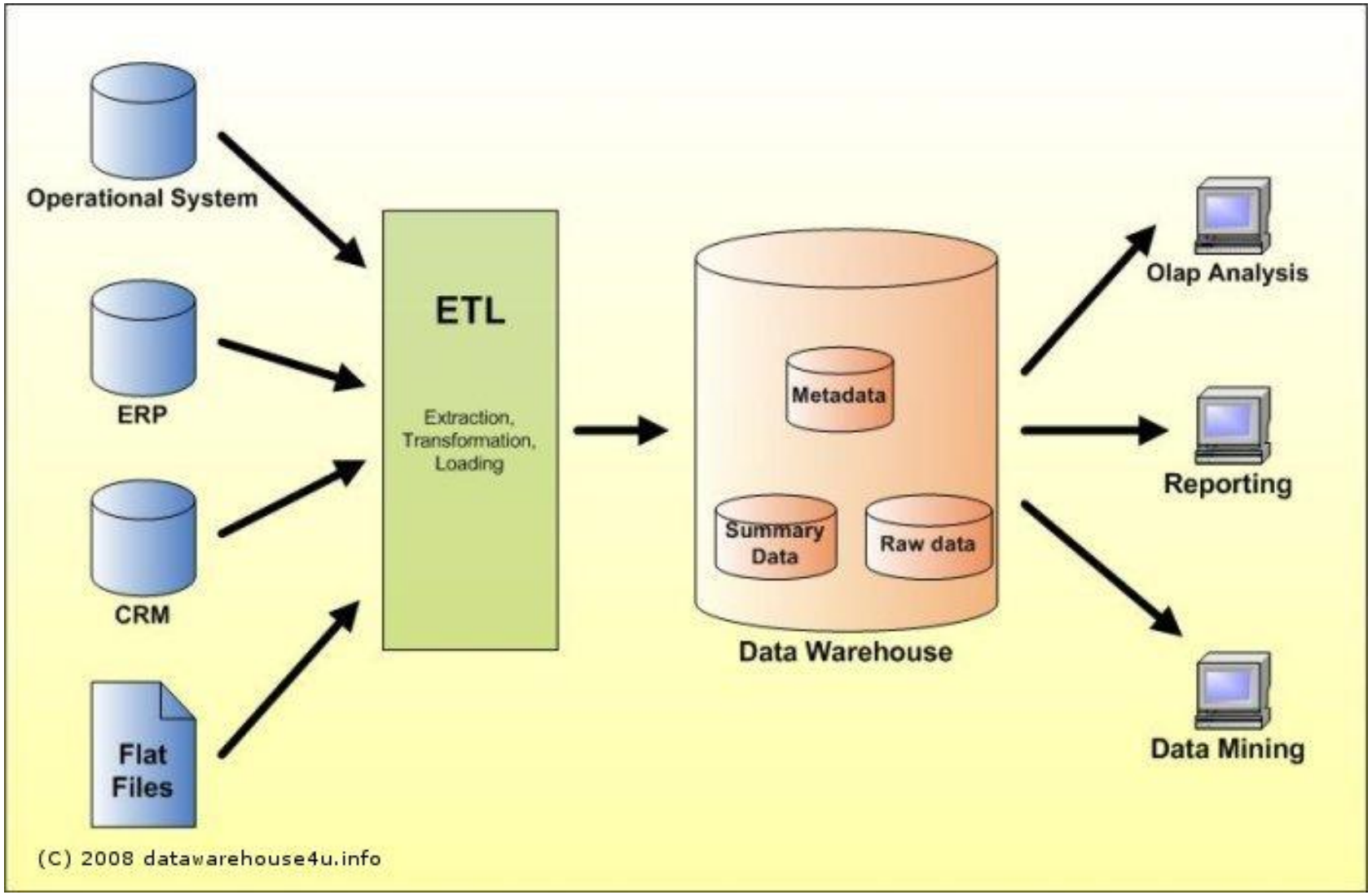
AI와 딥러닝을 활용한 경영분석

Week 3

Topics

- 데이터 전처리 (인코딩, 스케일링, 재샘플링)
- 세일 데이터 전처리 실습
- 대출 데이터 전처리 실습

Data Warehouse 데이터 창고



Extracting Data 데이터 추출

- Data from different source systems (SAP, ERP, other operational systems) is converted into one consolidated data warehouse format which is ready for transformation processing. 다른 소스(SAP, ERP, etc.)로부터 온 데이터들은 변환처리를 위해 하나의 통합된 데이터 창고 형식으로 전환

Transforming Data 데이터 변환

- Applying business rules (e.g., calculating new measures and dimensions). 비지니스 룰을 적용 (새로운 값을 계산)
- Cleaning (e.g., mapping NULL to 0 or “Male” to “M” and “Female” to " F " etc.). 데이터 클리닝 (널값처리, 값대체)
- Filtering (e.g., selecting only certain columns to load). 필터링(특정한 컬럼만 포함)

Transforming Data (cont.)

- Splitting a column into multiple columns and vice versa. 한개의 컬럼을 여러 개의 컬럼으로 나누거나 여러 개의 컬럼을 한 개의 컬럼으로 합침
- Joining together data from multiple sources(e.g., lookup, merge). 여러 개의 소스로부터 데이터를 하나로 합침
- Transposing rows and columns. 행과 열을 바꿈
- Applying any kind of simple or complex data validation (e.g., if the first 3 columns in a row are empty then reject the row from processing). 단순, 복잡한 데이터 검증을 적용

Loading Data 데이터 적재

- Load the data into a data warehouse or data repository for other reporting applications. 리포트를 위해 데이터를 데이터창고나 저장소로 옮김

Data Types 데이터 타입

- Data can be further classified as being quantitative or categorical. 데이터는 계량형 또는 범주형으로 나뉘어짐



Quantitative data are numerical values that indicate how much or how many. 계량형 데이터는 얼마나 많은지를 나타내는 숫자들

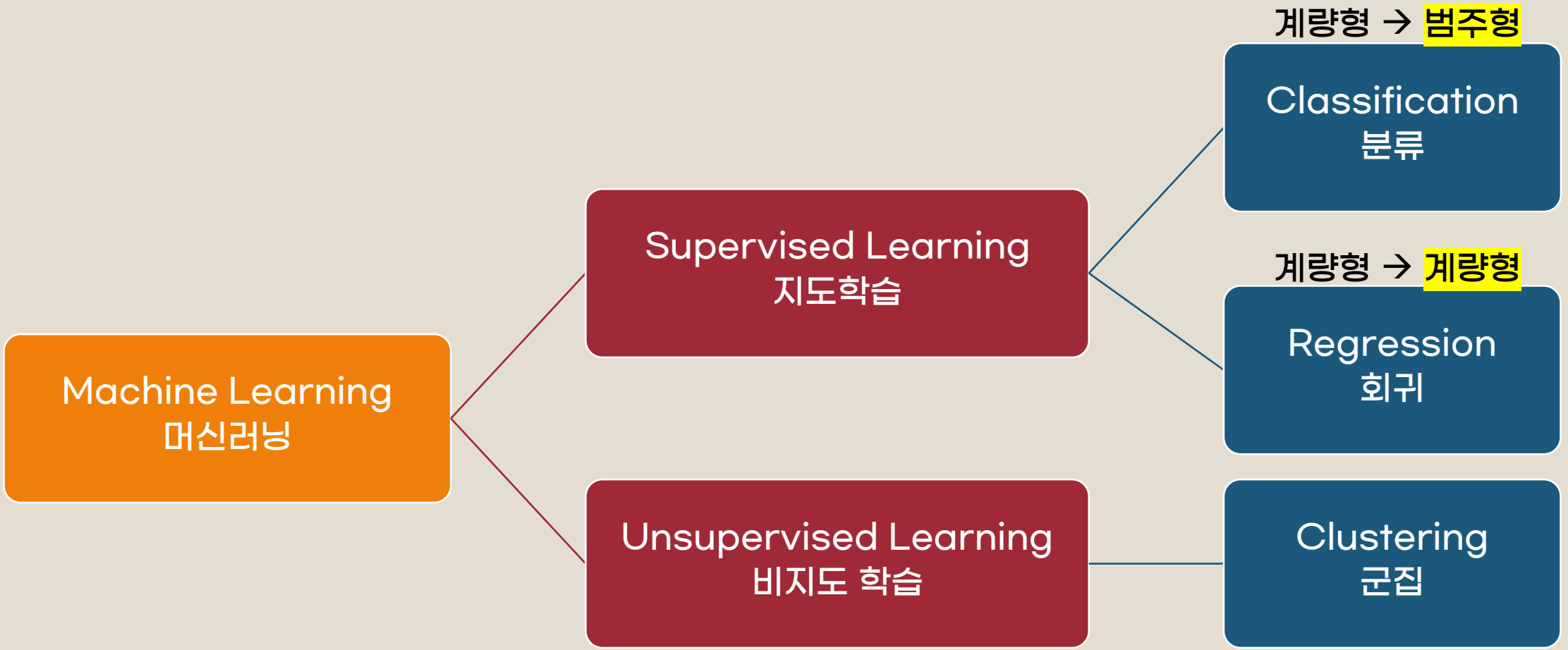
예) 키, 몸무게, 시험성적, 발생 횟수 등



Categorical data use labels or names to identify categories of like items. 범주형 데이터는 비슷한 아이템의 카테고리를 구분하는 레이블이나 이름을 사용하는 데이터들

예) 성별, 혈액형, 선호도 등

데이터 타입에 따른 머신러닝/인공지능 모델



Encoding 인코딩

- Encoding non-numeric values using: 범주형 데이터를 인코딩할 때 쓸 수 있는 방법들

replace	sklearn label encoder	sklearn one hot encoder	get_dummies

Replacing Single Value 한값 대체하기

- Use inplace=True to mutate the dataframe itself.
  데이터프레임 자체를 바꾸려면 inplace를 사용해야 함

```
df = pd.DataFrame({
    'name':['john','mary','paul'],
    'age':[30,25,40],
    'city':['new york','los angeles','london']
})
df.replace(25,40)
df.replace([30,25], np.nan)
```

Replacing Multiple Values
여러값 대체

- You can replace multiple values with dict 딕셔너리 형태를 이용하여 여러 값을 대체

```
df.replace({
    25:26,
    'john':'johnny'
})
```

Replacing with Regular Expressions
정규식을 이용한 대체

- The replacing rules apply to all columns. 모든 컬럼에 룰을 적용

```
df.replace(`[A-Za-z]','',regex=True)
```

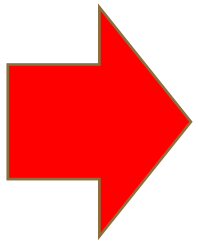
- When replacing values in multiple columns. 여러 개의 열에 있는 값 대체

```
df.replace({
   'temperature': '[A-Za-z]',
    'windspeed': '[A-Za-z]'},"", regex=True)
```

Replacing Categorical Values
범주형 값의 대체

```
df = pd.DataFrame({
    'score': ['exceptional', 'average', 'good', 'poor', 'average', 'exceptional'],
    'student':['rob', 'maya', 'parthiv', 'tom', 'julian', 'erica']
})
df = df.replace(['poor', 'average', 'good', 'exceptional'],[1,2,3,4])
```

Index	score	student
0	exceptional	kim
1	average	park
2	good	lee
3	poor	moon
4	average	choi
5	exceptional	yoon



Index	score	student
0	4	kim
1	2	park
2	3	lee
3	1	moon
4	2	choi
5	4	yoon

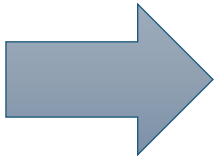
Lab #2

- Complete the replacing data tutorial on http://queirozf.com/entries/pandas-dataframe-replace-examples. 데이터를 대체하는 투토리얼을 끝내시오

Categorical Independent Variables
범주형 독립변수

- Examples 예
  - Gender (male, female), method of payment (cash, check, credit card), etc. 성별 (여자, 남자), 지불방법 (현금, 체크, 크레딧카드)
- Use the dummy or indicator variables. 범주형 변수의 경우 더미변수로 전환
  - Gender - 0 for male and 1 for female. 남자는 0, 여자는 1

Service Call	Months Since Last Service	Type of Repair	Repair Time in Hours
1	2	electrical	2.9
2	6	mechanical	3.0
3	8	electrical	4.8
4	3	mechanical	1.8
5	2	electrical	2.9
6	7	electrical	4.9
7	9	mechanical	4.2
8	8	mechanical	4.8
9	4	electrical	4.4
10	6	electrical	4.5



	A	B	C	D
1	Service Call	Months Since Last Service	Type of Repair	Repair Time in Hours
2	1	2	1	2.9
3	2	6	0	3.0
4	3	8	1	4.8
5	4	3	0	1.8
6	5	2	1	2.9
7	6	7	1	4.9
8	7	9	0	4.2
9	8	8	0	4.8
10	9	4	1	4.4
11	10	6	1	4.5

  - Type of Repair - 0 for mechanical, 1 for electrical. 기계적 결함은 0, 전기적 결함은 1

Complex Categorical Variables
복잡한 범주형 변수

- If a categorical variable has k levels, k - 1 dummy variables are required, with each dummy variable being coded as 0 or 1. 범주보다 한 개 적은 더미변수가 사용될 수 있음
- A variable with levels A, B, and C could be represented by x1 and x2 values of (0, 0) for A, (1, 0) for B, and (0,1) for C. 범주가 A,B,C이면 2개의 더미변수를 사용해서 A는 00으로, B는 10으로, C는 01로 나타낼 수 있음
- For example, a variable indicating level of education could be represented by x1 and x2 values as follows: 예를 들어 교육수준을 나타내는 변수를 x1과 x2로 표시

Highest Degree	x1	x2
Bachelor's	0	0
Master's	1	0
Ph.D	0	1

Label Encoder 라벨 인코더

```
# 라벨 인코더 이용하는 방법
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
johnson['TypeOfRepair'] = le.fit_transform(johnson.TypeOfRepair)

# replace 이용하는 방법
johnson_df = johnson.replace(['mechanical','electrical'], [0,1])
```

One Hot Encoder 원핫 인코더

```
# 원핫 인코더 이용하는 방법
From sklearn.preprocessing import OneHotEncoder
Ohe = OneHotEncoder()
Dummy = ohe.fit_transform(johnson.TypeOfRepair.values.reshape(-
1,1)).toarray()

# get_dummies 이용하는 방법
dummy = pd.get_dummies(johnson.TypeOfRepair)
```

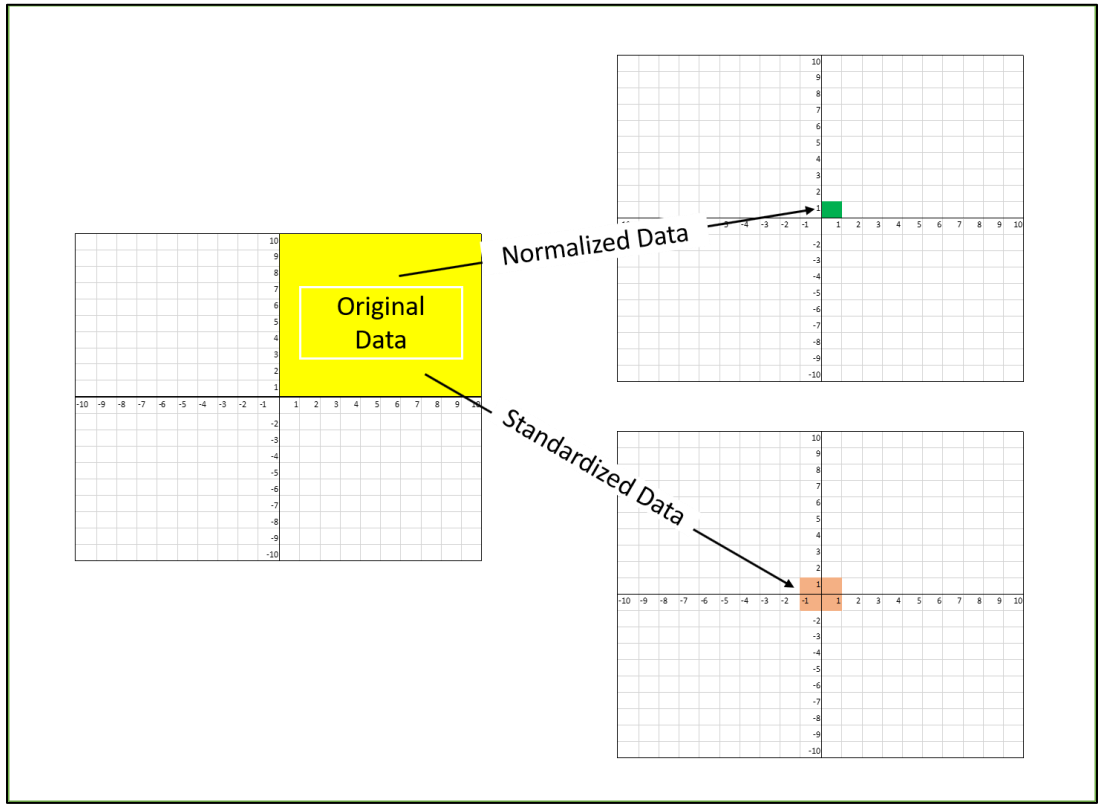
Lab #2

- Complete the encoding tutorial on https://medium.com/@contactsunny/label-encoder-vs-one-hot-encoder-in-machine-learning-3fc273365621 인코딩 투토리얼을 마치시오

Feature Scaling 변수 스케일링

- The range of all features should be normalized so that each feature contributes approximately proportionately to the final distance. 정규화를 하면 각 변수가 결과에 미치는 영향을 비례적으로 알 수 있음
- Since the range of values of raw data varies widely, in some machine learning algorithms, objective functions will not work properly without normalization. 정규화를 하지 않으면 제대로 작동하지 않은 목적함수가 있음
- Neural network algorithm converges faster with normalized features. 정규화를 하면 결과에 빨리 도착함

Types of Feature Scaling 종류

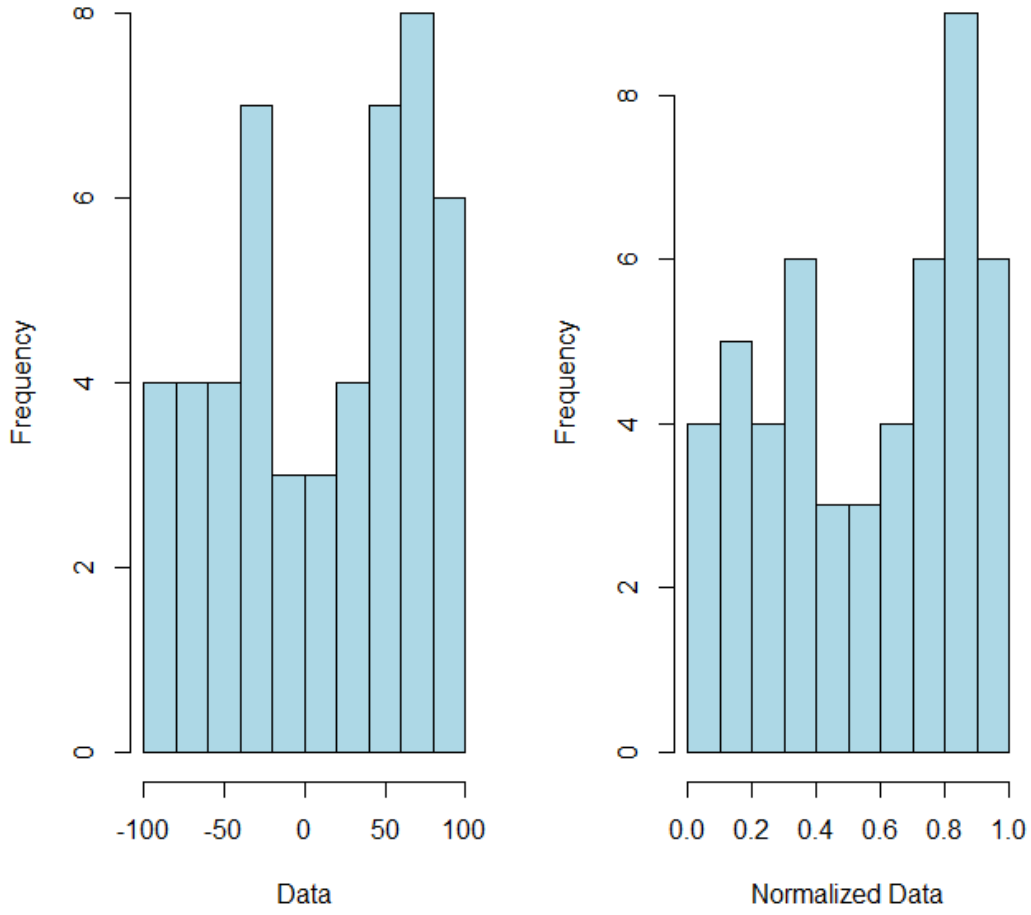


- Normalization - bound the values between two numbers. E.g., [0,1] or [-1,1]. 정규화는 두 숫자 사이의 값으로 변환하는 것
- Standardization - transforms zero mean and a variance of 1. 표준화는 평균이 0이고 분산이 1인 값으로 변환하는 것

Normalization Before Building Model
모델개발 전 정규화

$$z_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

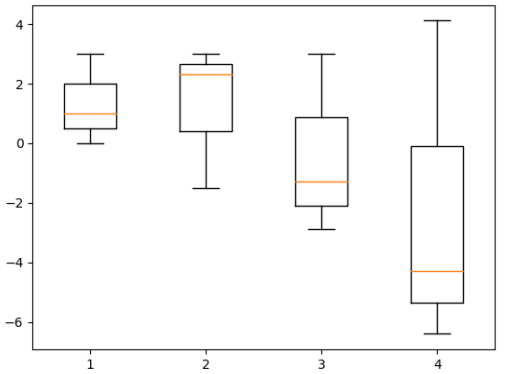
- Where x=(x1,...,xn) and zi is ith normalized data.

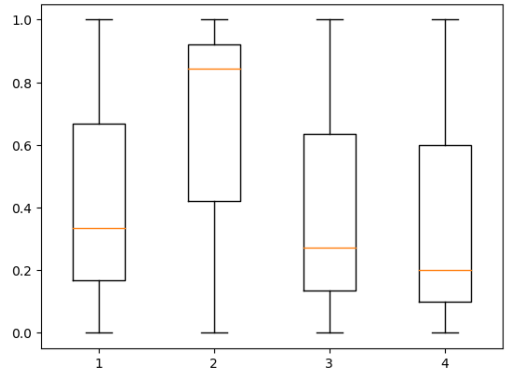


MinMaxScaler 최소 최대값 스케일러

- Transform features by scaling each feature to a given range. 각 속성을 주어진 범위로 스케일링 해서 변환

```
from sklearn.preprocessing import MinMaxScaler
input_data = np.array([[3, -1.5, 3, -6.4],
                       [0, 3, -1.3, 4.1],
                       [1, 2.3, -2.9, -4.3]])
scaler = MinMaxScaler(feature_range = (0, 1))
data_scaled = scaler.fit_transform(data)
```

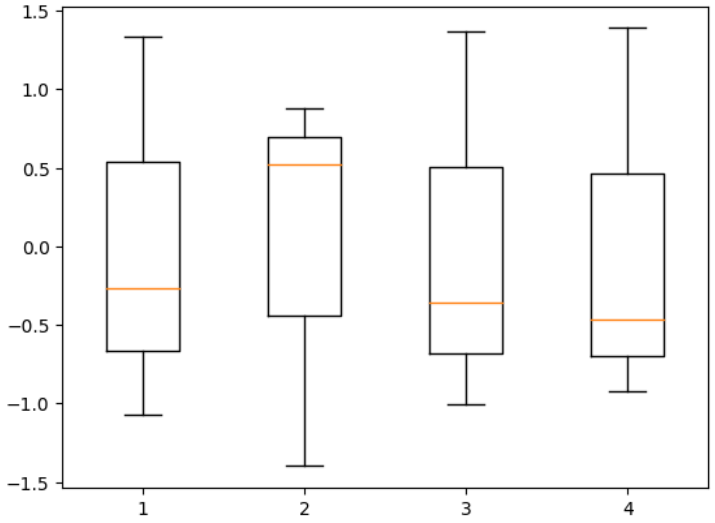




StandardScaler 표준 스케일러

- Transforms attributes with a Gaussian distribution and differing means and standard deviations to a standard Gaussian distribution with a mean of 0 and a standard deviation of 1. 평균0이 00이고 표준 편차가 1인 표준 가우스 분포로 변환

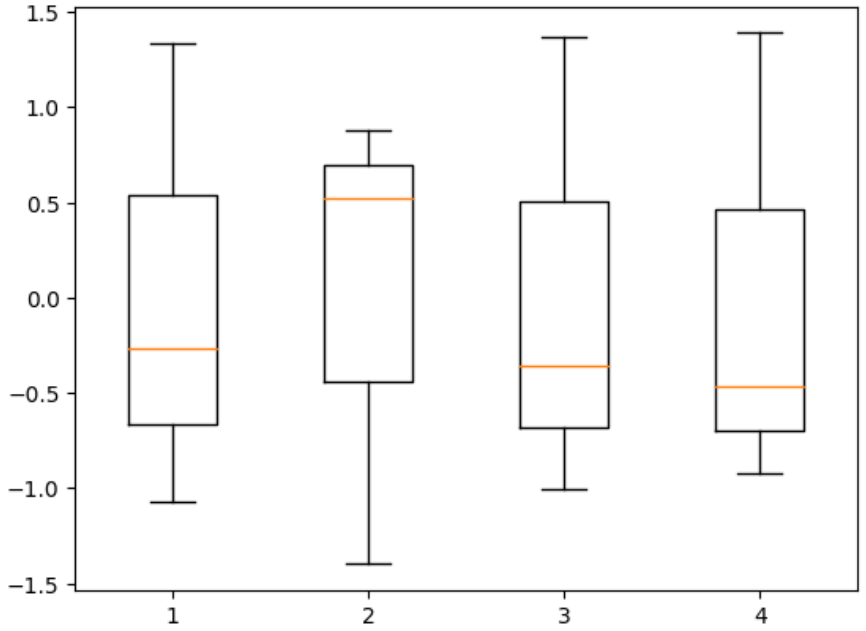
```
from sklearn.preprocessing import StandardScaler
scalar = StandardScaler()
data_scaled = scaler.transform(data)
```



Mean Removal 평균제거

- It involves removing the mean from each feature so that it is centered on zero. 각 속성에서 평균을 빼기 때문에 0이 중앙이 됨
- Removes any bias from the features. 속성에서 바이어스를 제거

```
import numpy as np
from sklearn.preprocessing import scale
data_scaled = scale(data)
```



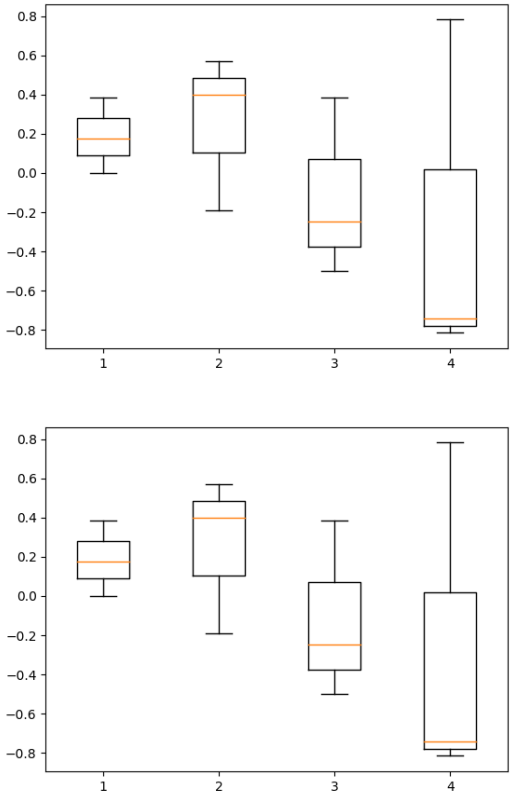
Normalization 정규화

- Scale each observation (row) to have a length of 1. 각 줄을 스케일해서 길이가 1이 되게 함
- Useful for sparse datasets (lots of zeros). 0이 많은 데이터셋에 유용

```
from sklearn.preprocessing import normalize,
Normalizer

# 함수형 정규화
Data_scaled = normalize(data, norm  = 'l2')
#l1 uses absolute value, l2 uses square root

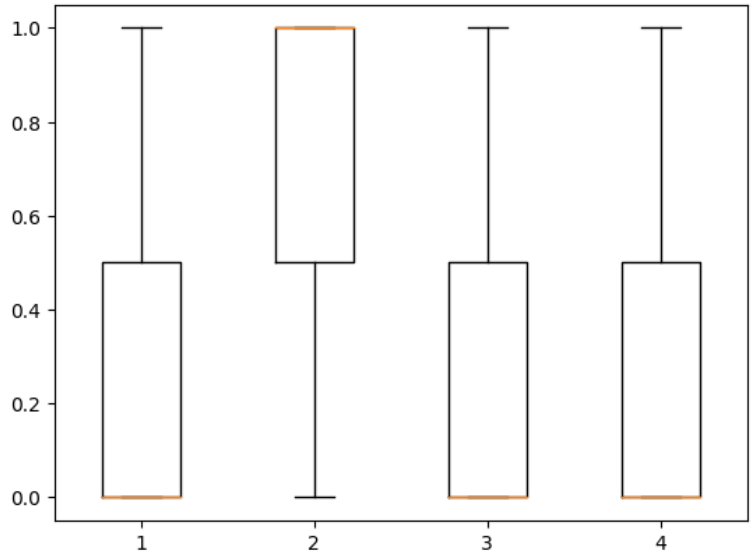
# 객체형 정규화
Scaler = Normalizer() #기본값이 L2
data_normalized = scaler.fit_transform(input_data)
```



Binarization 이진화

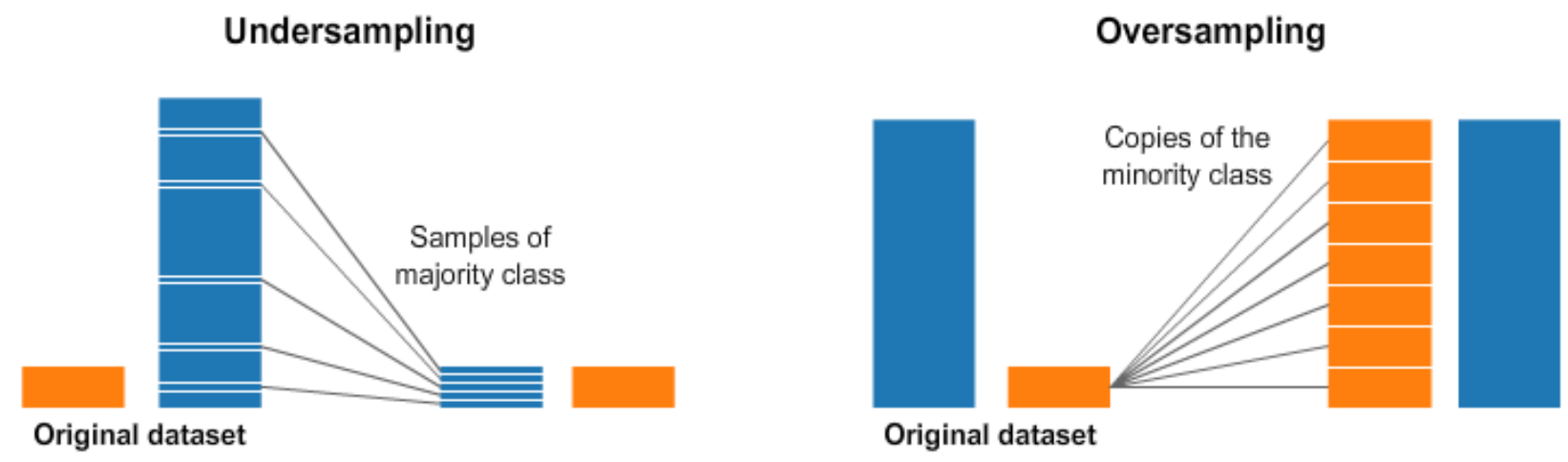
- Used to convert a numerical feature vector into a Boolean vector. 숫자벡터를 불린벡터로 변환

```
from sklearn.preprocessing import Binarizer
data_scaled =
Binarizer(threshold=1.4).transform(data)
```



Resampling 재샘플링

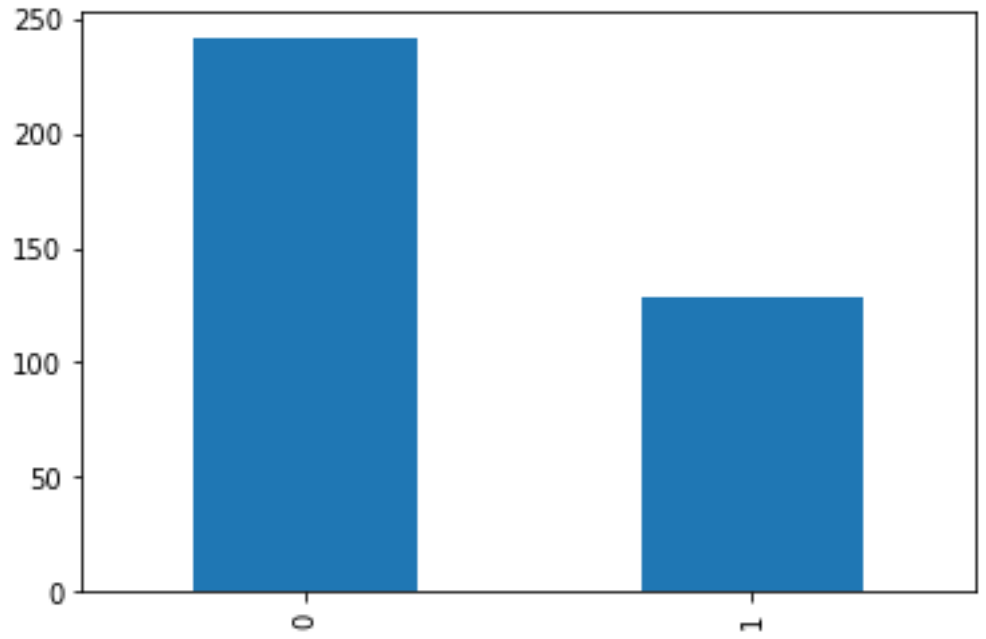
- A technique for dealing with highly unbalanced datasets. 불균형 데이터 세트를 처리하기 위한 기술
- Removing samples from the majority class (under-sampling) and / or adding more examples from the minority class (over-sampling). 다수의 클래스에서 샘플 제거 (언더샘플링) 하거나 소수의 클래스(오버샘플링)에서 더 많은 샘플 추가(오버샘플링)



Python Imbalanced-Learn Module

- A python package offering a number of re-sampling techniques commonly used in datasets. 데이터 세트에 일반적으로 사용되는 여러 가지 재샘플링 기술을 제공하는 파이썬 패키지

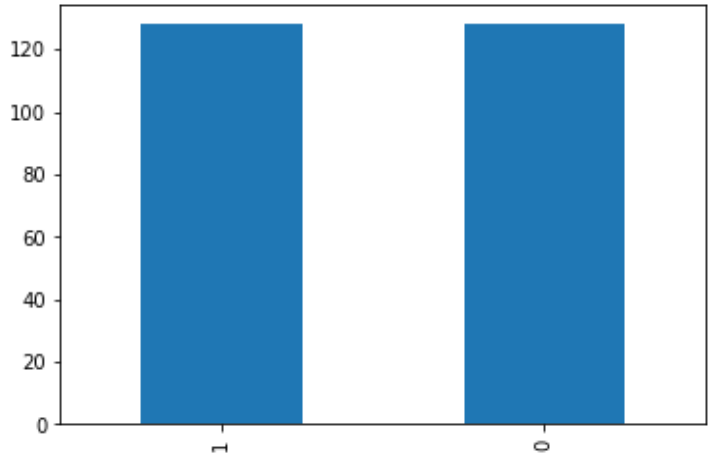
```
import imblearn
```



Random Under-Sampling
랜덤 언더샘플링

```
from imblearn.under_sampling import
RandomUnderSampler

rus =
RandomUnderSampler(return_indices=True)
X_rus, y_rus = rus.fit_sample(X, y)
```

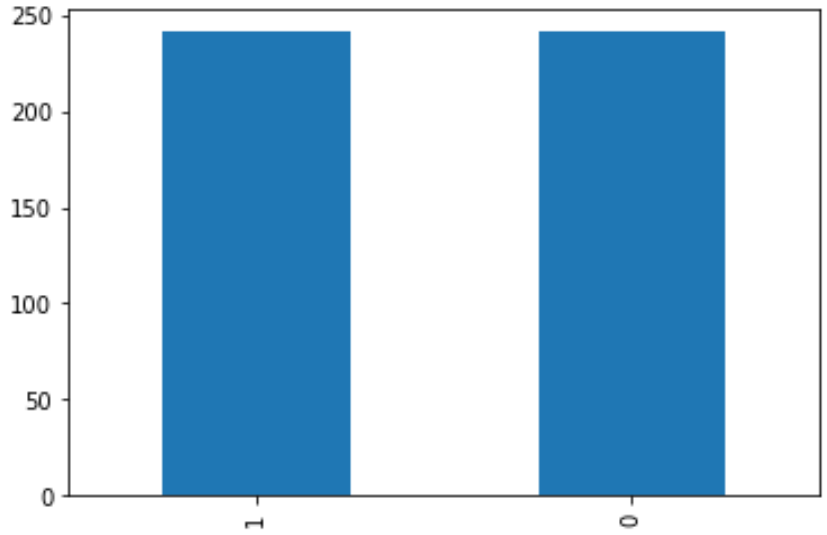


Random Over-Sampling
랜덤 오버샘플링

- 실패케이스를 복사하여 같은 갯수로 만듬

```
from imblearn.over_sampling import
RandomOverSampler

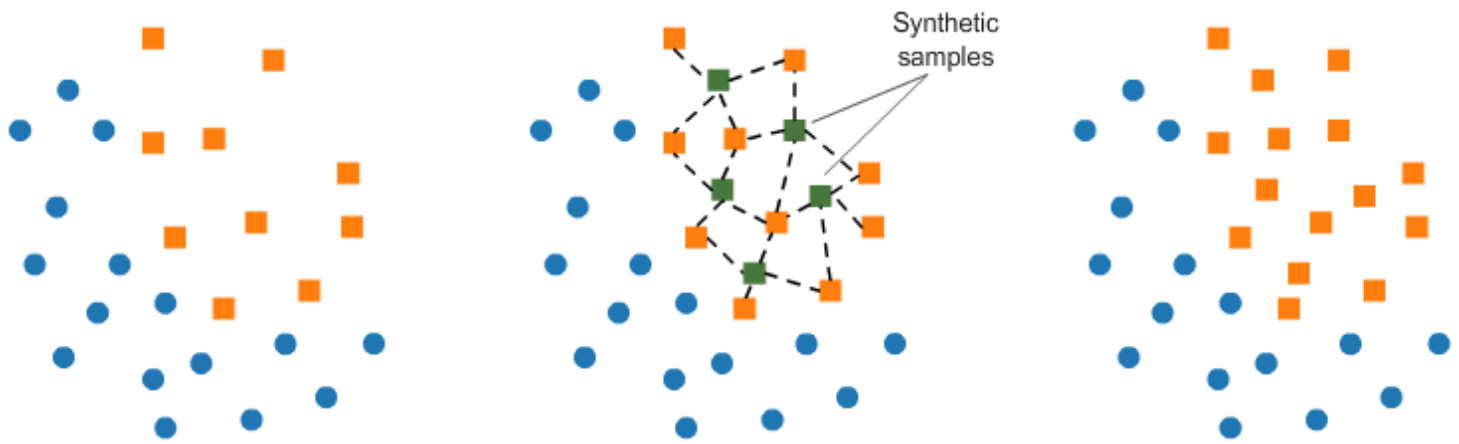
ros = RandomOverSampler()
X_ros, y_ros = ros.fit_sample(X, y)
```



SMOTE (Synthetic Minority Oversampling Technique)

- Works randomly pick a point from the minority class and computing the k-nearest neighbors for this point. The synthetic points are added between the chosen point and its neighbors. 소수클래스에서 무작위로 한 점을 선택하고 이 점에 대해 가장 가까운 이웃을 계산. 합성점은 선택한 점과 인접 지점 사이에 추가

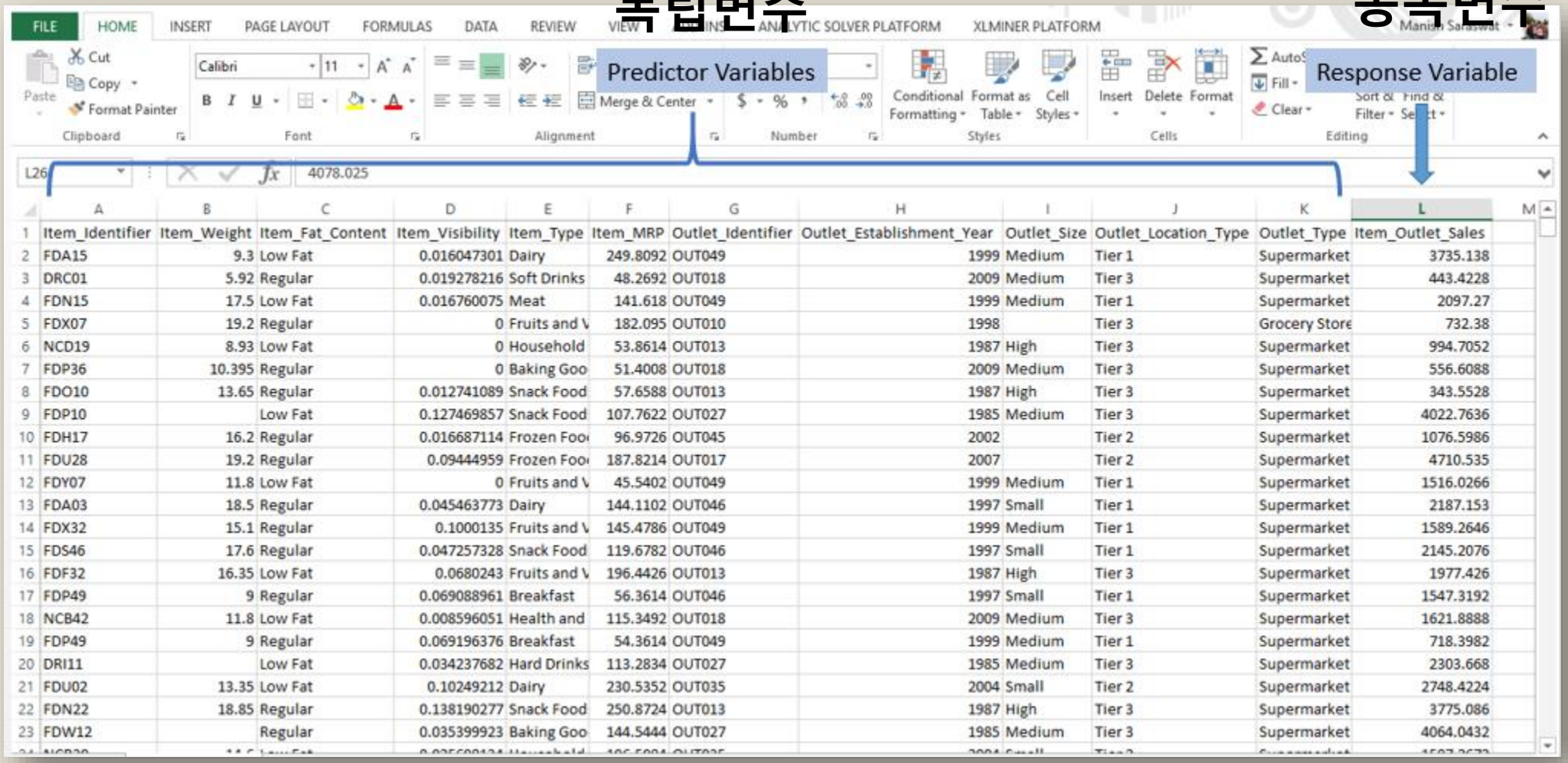
```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=27)
x_train, y_train = smote.fit_sample(x_train, y_train)
```



Big Mart Sales Data Set 데이터셋

독립변수

종속변수



	A	B	C	D	E	F	G	H	I	J	K	L
1	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type	Outlet_Type	Item_Outlet_Sales
2	FDA15	9.3	Low Fat	0.016047301	Dairy	249.8092	OUT049	1999	Medium	Tier 1	Supermarket	3735.138
3	DRC01	5.92	Regular	0.019278216	Soft Drinks	48.2692	OUT018	2009	Medium	Tier 3	Supermarket	443.4228
4	FDN15	17.5	Low Fat	0.016760075	Meat	141.618	OUT049	1999	Medium	Tier 1	Supermarket	2097.27
5	FDX07	19.2	Regular	0	Fruits and V	182.095	OUT010	1998		Tier 3	Grocery Store	732.38
6	NCD19	8.93	Low Fat	0	Household	53.8614	OUT013	1987	High	Tier 3	Supermarket	994.7052
7	FDP36	10.395	Regular	0	Baking Goo	51.4008	OUT018	2009	Medium	Tier 3	Supermarket	556.6088
8	FDO10	13.65	Regular	0.012741089	Snack Food	57.6588	OUT013	1987	High	Tier 3	Supermarket	343.5528
9	FDP10		Low Fat	0.127469857	Snack Food	107.7622	OUT027	1985	Medium	Tier 3	Supermarket	4022.7636
10	FDH17	16.2	Regular	0.016687114	Frozen Foo	96.9726	OUT045	2002		Tier 2	Supermarket	1076.5986
11	FDU28	19.2	Regular	0.09444959	Frozen Foo	187.8214	OUT017	2007		Tier 2	Supermarket	4710.535
12	FDY07	11.8	Low Fat	0	Fruits and V	45.5402	OUT049	1999	Medium	Tier 1	Supermarket	1516.0266
13	FDA03	18.5	Regular	0.045463773	Dairy	144.1102	OUT046	1997	Small	Tier 1	Supermarket	2187.153
14	FDX32	15.1	Regular	0.1000135	Fruits and V	145.4786	OUT049	1999	Medium	Tier 1	Supermarket	1589.2646
15	FDS46	17.6	Regular	0.047257328	Snack Food	119.6782	OUT046	1997	Small	Tier 1	Supermarket	2145.2076
16	FDF32	16.35	Low Fat	0.0680243	Fruits and V	196.4426	OUT013	1987	High	Tier 3	Supermarket	1977.426
17	FDP49	9	Regular	0.069088961	Breakfast	56.3614	OUT046	1997	Small	Tier 1	Supermarket	1547.3192
18	NCB42	11.8	Low Fat	0.008596051	Health and	115.3492	OUT018	2009	Medium	Tier 3	Supermarket	1621.8888
19	FDP49	9	Regular	0.069196376	Breakfast	54.3614	OUT049	1999	Medium	Tier 1	Supermarket	718.3982
20	DRI11		Low Fat	0.034237682	Hard Drinks	113.2834	OUT027	1985	Medium	Tier 3	Supermarket	2303.668
21	FDU02	13.35	Low Fat	0.10249212	Dairy	230.5352	OUT035	2004	Small	Tier 2	Supermarket	2748.4224
22	FDN22	18.85	Regular	0.138190277	Snack Food	250.8724	OUT013	1987	High	Tier 3	Supermarket	3775.086
23	FDW12		Regular	0.035399923	Baking Goo	144.5444	OUT027	1985	Medium	Tier 3	Supermarket	4064.0432

Variables 변수들

Variable	Description
Item_Identifier 아이템 아이디	Unique product ID
Item_Weight 아이템 무게	Weight of product
Item_Fat_Content 지방함유	Whether the product is low fat or not
Item_Visibility 아이템 노출도	% of total display area in store allocated to this product
Item_Type 아이템 종류	Category to which product belongs
Item_MRP 아이템 가격	Maximum Retail Price (list price) of product
Outlet_Identifier 아웃렛 ID	Unique store ID
Outlet_Establishment_Year 설립년도	Year in which store was established
Outlet_Size 아웃렛사이즈	Size of the store
Outlet_Location_Type 위치종류	Type of city in which store is located
Outlet_Type 아웃렛종류	Grocery store or some sort of supermarket
Item_Outlet_Sales 아웃렛세일	Sales of product in particular store. This is the outcome variable to be predicted.

Lab #2 Big Mart Sales Data Preprocessing

- Import the train and test data of Big Mart Sales and do some basic exploration tasks. 훈련, 테스트용 데이터를 임포트한 후에 기본 데이터탐색을 수행하시오.
  - read_csv() 파일읽기
  - shape, dtypes, describe() 데이터검색
  - df.isnull().sum() 널값검색

Lab #2 Big Mart Sales Data Preprocessing

- Set a value 1 for outlet sales in the test dataset.
  테스트테이터에 아웃렛세일 열을 만들고 1을 할당
- Combine train and test data. 훈련과 테스트데이터를 합침

Lab #2 Big Mart Sales Data Preprocessing

- Impute the missing values and assign a value to mis-matched levels. 분실값 및 매칭되지 않는 값 대체
  - Impute missing values by median for item weight. 아이템 무게의 분실값을 중위수로 대체
  - Impute missing values by median for item visibility. 노출도의 분실값을 중위수로 대체
  - Assign the name “Other” to unnamed level in outlet size variable. 아웃렛 사이즈의 분실값을 Other로 대체
  - Rename the various levels of item fat content. 지방함유율의 값 대체
    - LF to Low Fat, reg to Regular, low fat to Low Fat
- Create a new column, Year = 2026 - outlet establishment year. 새로운 열 만들기

Lab #2 Big Mart Sales Data Preprocessing

- Drop the variables not required in prediction model (exclude item identifier, outlet identifier, item fat content, outlet establishment year, and item type). 필요없는 열을 삭제
- Encode the categorical variables. 범주형 변수를 인코딩
- X and y split. 독립변수, 종속변수 분리
- Train and test split. 훈련, 테스트 데이터 분리

Train and test split 훈련, 테스트데이터 분리

```
from sklearn import model_selection
x_train, x_test, y_train, y_test =
model_selection.train_test_split(x, y, test_size=.2,
random_state=7) #데이터를 나눠서 80프로는 훈련용으로 20프로는
테스트용으로 사용
```

Loan Prediction Problem
대출예측문제

- Company wants to automate the loan eligibility process (real time) based on customer detail provided while filling online application form. These details are Gender, Marital Status, Education, Number of Dependents, Income, Loan Amount, Credit History and others. To automate this process, they have given a problem to identify the customers segments, those are eligible for loan amount so that they can specifically target these customers. 고객이 온라인신청서를 제출할때 제공하는 정보를 바탕으로 대출승인절차를 자동화하려고 합니다. 성별, 결혼여부, 교육, 자녀의 수, 수입, 대출금액, 크레딧히스토리 등의 정보가 포함됩니다. 대출자격이 되는 고객을 찾는 모델을 만드십시오.

Variable Descriptions 변수 설명

	Variable 변수	Description 설명
Non-numeric 범주형	Loan_ID 아이디	Unique Loan ID
	Gender 성별	Male/ Female
	Married 결혼여부	Applicant married (Y/N)
	Dependents 부양가족수	Number of dependents
	Education 교육정도	Applicant Education (Graduate/ Under Graduate)
	Self_Employed 자영업자	Self employed (Y/N)
Numeric 계량형	ApplicantIncome 수입	Applicant income
	CoapplicantIncome 보증인수입	Coapplicant income
	LoanAmount 대출금	Loan amount in thousands
	Loan_Amount_Term 대출기간	Term of loan in months
Non-numeric 범주형	Credit_History 신용기록	credit history meets guidelines
	Property_Area 부동산지역	Urban/ Semi Urban/ Rural
	Loan_Status 대출여부	Loan approved (Y/N)

Lab #2 Loan Prediction Data Preparation

- Import the train data of Loan Predication and do some basic exploration tasks. 대출 훈련데이터를 가져와서 기본 탐색을 수행하시오.
  - head(), shape, dtypes, describe(), value_counts()

Lab #2 Loan Prediction Data Preparation

- Preprocess the data to apply classification algorithms. 데이터 전처리
  - Missing values 분실값처리
    - Loan amount → mean
    - Gender, married, dependents, loan amount term, credit history → mode
  - Encoding 인코딩
    - Gender, married, dependents, education, self employed, property area, loan status

Missing values 분실값 처리

- Combine train and test data and fill missing data. 훈련과 테스트 데이터를 합친 후 분실값 처리
  - Mean for Loan_Amount. 대출액은 평균으로 대체
  - Mode for Self_Employed, Gender, Married, Dependents, Loan_Amount_Term, and Credit_History. 나머지는 최빈수로 대체

- `df.isna().sum()`
- `value_counts()` → `fillna()`
- `fillna(mean(), mode(), …., inplace=True)`

Encoding 인코딩

- Encoding non-numeric values.
  범주형 변수를 인코딩
  - Use get_dummies for Property_Area
  - Use label encoder for Gender, Married, Education, Self_Employed (can use for loop)
  - Use replace for Loan_Status ({'Y':1, 'N':0}, inplace=True) and Dependents ('3+','3', inplace=True) and change the data type to int32 using astype
- Dummy variables
  - pd.get_dummies() → pd.concat() → df.drop()
- LabelEncoder in sklearn.preprocessing
  - le = LabelEncoder()
  - le.fit_transform()
- OneHotEncoder in sklearn.preprocessing
  - ohe = OneHotEncoder()
  - ohe.fit_transform()

Lab #2 Loan Prediction Data Preparation

- X and y split. 독립변수, 종속변수 분리
  - Loan_Status for dependent variables.
  - Other variables (excluding 'Loan_ID', 'Loan_Status', 'Property_Area‘) for independent variables.

Lab #2 Loan Prediction Data Preparation

- Train and test split. 훈련, 테스트 데이터 분리

```
from sklearn import model_selection
x_train, x_test, y_train, y_test =
model_selection.train_test_split(x, y, test_size=.2,
random_state=7) #데이터를 나눠서 80프로는 훈련용으로 20프로는
테스트용으로 사용
```